Trabalho AV-2 – QuickSort: implementação e análise de complexidade

1 INTRODUÇÃO

Este trabalho trata do problema de ordenar um vetor de n números inteiros em ordem crescente. Os vetores usados nos testes são permutações aleatórias dos números de 1 até n, então cada número aparece uma única vez e não há valores repetidos.

O algoritmo utilizado é o QuickSort, que segue a ideia de divisão e conquista. Primeiro é escolhido um elemento do vetor, chamado de pivô. Em seguida o vetor é particionado: os elementos menores que o pivô ficam à esquerda dele, os maiores ficam à direita, e o próprio pivô já fica na posição em que estará no vetor ordenado. Depois o mesmo processo é aplicado ao lado esquerdo e ao lado direito, até que os pedaços tenham tamanho 0 ou 1.

O tempo gasto pelo QuickSort depende de como os dois lados ficam divididos, e isso depende da escolha do pivô. Por esse motivo foram feitas duas versões, as duas com o mesmo procedimento de particionamento:

i. QuickSort com pivô aleatório, em que o pivô é sorteado entre os elementos do pedaço;
ii. QuickSort Proporcional, que recebe um parâmetro α, com $0 < \alpha \leq 1/2$, e escolhe o pivô de modo que um pedaço de tamanho n seja sempre dividido em partes de tamanho αn e $(1 - \alpha)n$. Para a análise foram escolhidos $\alpha = 1/2$ (divisão 50%–50%) e $\alpha = 1/4$ (divisão 25%–75%).

O programa foi feito em um único arquivo, `QuickSort.java`. Para cada tamanho n (1.000, 10.000, 100.000 e 1.000.000), o `main` gera um vetor com uma permutação aleatória de 1 a n, ordena esse vetor, confere se o resultado ficou igual a $1, 2, \ldots, n$ e repete isso cinco vezes, mostrando o tempo médio. Isso é feito para o QuickSort aleatório e para o Proporcional com $\alpha = 0{,}5$ e $\alpha = 0{,}25$.

Para compilar e executar:

```
javac QuickSort.java
java QuickSort
```

2 IMPLEMENTAÇÃO

2.1 Estruturas de dados e TAD

A estrutura usada é um vetor de inteiros (`int[]`). A ordenação é feita no próprio vetor, sem usar vetor auxiliar. Um trecho do vetor é indicado por dois índices, `ini` e `fim`, ou seja, o subvetor `v[ini..fim]`. Os trechos que ainda precisam ser ordenados ficam guardados na própria pilha de recursão do Java. Também é usado um objeto `Random`, da biblioteca do Java, para sortear o pivô e embaralhar os vetores.

O vetor junto com as funções que operam sobre ele foi tratado como um TAD (Vetor de Inteiros). Os dados são uma sequência $v[0..n-1]$ de inteiros distintos, e as operações estão descritas na tabela abaixo.

Operação	**Pré-condição**	**Pós-condição**
`gerarVetor(n)`	$n \geq 1$	retorna uma permutação aleatória de $1..n$
`trocar(v, i, j)`	`i` e `j` são posições válidas	os valores de `v[i]` e `v[j]` foram trocados
`particionar(v, ini, fim)`	`ini ≤ fim`	retorna `q` tal que `v[ini..q-1] ≤ v[q] < v[q+1..fim]`
`quickSortAleatorio(v, ini, fim)`	—	`v[ini..fim]` está em ordem crescente
`quickSortProporcional(v, ini, fim, alfa)`	$0 < \alpha \leq 1/2$ e `v` é permutação de $1..n$	`v[ini..fim]` está em ordem crescente
`estaOrdenado(v)`	—	retorna `true` somente se `v` é $1, 2, \ldots, n$

2.2 Funções do programa

Função	**Descrição**
`particionar`	Particionamento de Lomuto. Usa `v[fim]` como pivô e retorna a posição final dele.
`trocar`	Troca os valores de duas posições do vetor.
`quickSortAleatorio`	Versão i. Sorteia o pivô, particiona e ordena os dois lados.
`quickSortProporcional`	Versão ii. Escolhe o pivô que divide o trecho na proporção $\alpha : (1 - \alpha)$.
`gerarVetor`	Gera uma permutação aleatória de 1 a n (embaralhamento de Fisher-Yates).
`estaOrdenado`	Verifica se o vetor ficou $1, 2, \ldots, n$.
`main`	Executa os testes e mostra os resultados.

2.3 Particionamento de Lomuto

No Lomuto o pivô é sempre o último elemento do trecho, `v[fim]`. O índice `i` marca até onde vai a região dos elementos menores ou iguais ao pivô (em aula chamamos essa região de “cercadinho”), e começa em `ini - 1`, ou seja, com a região vazia. O índice `j` percorre o trecho de `ini` até `fim - 1`. Sempre que `v[j]` é menor ou igual ao pivô, `i` aumenta uma posição e `v[j]` é trocado com `v[i]`, entrando na região. No final, o pivô é trocado com `v[i+1]`, que é a posição correta dele.

Durante o laço o trecho fica dividido assim:

```
ini          i   i+1          j            fim
|  <= pivo   |   > pivo       |  nao visto  | pivo |
```

Exemplo com `v = [3, 8, 2, 5, 1, 4, 7, 6]` e pivô 6:

`j`	`v[j]`	≤ 6**?**	**ação**	**vetor**
0	3	sim	`i = 0`, troca(0,0)	`[3, 8, 2, 5, 1, 4, 7, 6]`
1	8	não	—	`[3, 8, 2, 5, 1, 4, 7, 6]`
2	2	sim	`i = 1`, troca(1,2)	`[3, 2, 8, 5, 1, 4, 7, 6]`
3	5	sim	`i = 2`, troca(2,3)	`[3, 2, 5, 8, 1, 4, 7, 6]`
4	1	sim	`i = 3`, troca(3,4)	`[3, 2, 5, 1, 8, 4, 7, 6]`
5	4	sim	`i = 4`, troca(4,5)	`[3, 2, 5, 1, 4, 8, 7, 6]`
6	7	não	—	`[3, 2, 5, 1, 4, 8, 7, 6]`
fim			troca(5,7)	`[3, 2, 5, 1, 4, 6, 7, 8]`

O pivô termina na posição $q = 5$, com 5 elementos à esquerda e 2 à direita. Foram feitas 7 comparações, isto é, $n - 1$.

2.4 Escolha entre Lomuto e Hoare

Foi escolhido o particionamento de Lomuto. O principal motivo é que no Lomuto o pivô termina na sua posição definitiva, e o trecho fica dividido em `v[ini..q-1]` e `v[q+1..fim]`, com exatamente k e $n - 1 - k$ elementos. Isso corresponde diretamente à recorrência $T(n) = T(k) + T(n - 1 - k) + cn$ usada na análise.

Essa propriedade também é necessária para a versão Proporcional, porque para forçar a divisão $\alpha : (1 - \alpha)$ basta escolher o pivô certo. No Hoare o pivô não fica necessariamente na posição final e o ponto de divisão depende da disposição dos dados, o que dificultaria controlar o tamanho de cada lado. Além disso, o Lomuto tem um único laço e faz sempre $n - 1$ comparações, o que facilita a análise.

O Lomuto tem desempenho ruim quando existem muitos valores repetidos e costuma fazer mais trocas que o Hoare. Como os vetores deste trabalho não têm repetidos, o primeiro problema não ocorre, e o número maior de trocas altera apenas a constante, não a complexidade.

2.5 Versão i: pivô aleatório

É sorteada uma posição `p` entre `ini` e `fim` (`p = ini + rand.nextInt(fim - ini + 1)`). Em seguida `v[p]` é trocado com `v[fim]`, porque o Lomuto sempre usa o último elemento como pivô. Depois disso o trecho é particionado e o algoritmo continua nos dois lados.

2.6 Versão ii: QuickSort Proporcional

Para um trecho de n elementos, o lado esquerdo deve ficar com $k = \lfloor \alpha(n-1) \rfloor$ elementos e o lado direito com $n - 1 - k$ (o pivô não entra em nenhum dos lados). Para isso o pivô tem que ser o $(k+1)$-ésimo menor elemento do trecho.

Como o vetor é uma permutação de 1 a n, depois de cada particionamento o trecho `v[ini..fim]` contém exatamente os valores que vão ocupar essas posições no vetor ordenado. No vetor ordenado a posição x tem o valor $x + 1$ (os índices começam em 0 e os valores em 1), então o trecho contém os valores de `ini + 1` até `fim + 1`. O $(k+1)$-ésimo menor é, portanto, o valor `ini + 1 + k`. O programa procura esse valor com uma busca linear, troca com `v[fim]` e particiona.

Por exemplo, para um vetor de 1 a 8 (`ini = 0`, `fim = 7`) e $\alpha = 1/4$, temos $k = \lfloor 0{,}25 \cdot 7 \rfloor = 1$ e o pivô é o valor 2. Depois do particionamento fica 1 elemento à esquerda e 6 à direita.

2.7 Tratamento do StackOverflowError

Na forma tradicional do QuickSort há duas chamadas recursivas. Quando as divisões são ruins, como 0 e $n-1$, cada chamada gera outra com apenas um elemento a menos, e a pilha chega a ter n chamadas. Para n grande (dezenas de milhares) isso causa `StackOverflowError`.

A solução adotada foi fazer a chamada recursiva apenas no lado menor e tratar o lado maior no próprio laço `while`, atualizando `ini` ou `fim`:

```
if (q - ini < fim - q) {
    quickSortAleatorio(v, ini, q - 1);
    ini = q + 1;
} else {
    quickSortAleatorio(v, q + 1, fim);
    fim = q - 1;
}
```

O teste `q - ini < fim - q` compara o tamanho do lado esquerdo (`q - ini` elementos) com o do lado direito (`fim - q` elementos). Como o lado menor tem no máximo metade dos elementos, cada chamada recursiva recebe no máximo metade do trecho anterior, e a pilha fica limitada a cerca de $\log_2 n$ chamadas (em torno de 20 para $n = 1.000.000$), mesmo no pior caso. O número de operações continua o mesmo; só o uso de memória da pilha cai de $O(n)$ para $O(\log n)$.

2.8 Decisões sobre casos omissos

A forma de achar o pivô do Proporcional (valor `ini + 1 + k` e busca linear) só funciona porque a entrada é uma permutação de 1 a n, o que é garantido pelo enunciado. A busca custa $\Theta(n)$, o mesmo que o particionamento, então ela só aumenta a constante c e não muda a complexidade.

O valor de k é calculado com `(int)`, que descarta a parte decimal, ou seja, arredonda para baixo. Como $\alpha \leq 1/2$, o lado esquerdo nunca fica maior que o direito.

Em trechos pequenos não é possível dividir exatamente na proporção α. Com $\alpha = 1/4$ e $n = 3$, por exemplo, $k = 0$ e a divisão fica 0 e 2. Isso só acontece quando $\alpha(n-1) < 1$, ou seja, em trechos com menos de $1/\alpha + 1$ elementos (menos de 5 para $\alpha = 1/4$). Esses trechos aparecem apenas nos últimos níveis da recursão e têm tamanho constante, então não alteram a complexidade.

Foram usados somente $\alpha = 1/2$ e $\alpha = 1/4$. A divisão 90%–10% não foi usada, conforme pedido no enunciado.

2.9 Entrada e saída

O programa não lê dados do usuário. Os vetores são gerados pela função `gerarVetor`, que preenche o vetor com $1, 2, \ldots, n$ e embaralha, nos tamanhos 1.000, 10.000, 100.000 e 1.000.000. Para cada tamanho e cada versão, a saída mostra o tempo médio de cinco execuções, em milissegundos (medido com `System.nanoTime()`), e se o vetor ficou ordenado (`true` ou `false`).

3 ANÁLISE DE COMPLEXIDADE

Nesta seção, $T(n)$ é o tempo para ordenar um trecho de n elementos, $P(n)$ é o tempo do particionamento e c é uma constante positiva.

3.1 i. Procedimento de particionamento

Considerando um trecho com $n = fim - ini + 1$ elementos, cada linha do `particionar` tem um custo constante c_i e é executada um certo número de vezes:

Linha	Código	Custo	Vezes
1	`pivo = v[fim]`	c_1	1
2	`i = ini - 1`	c_2	1
3	`for (j = ini; j < fim; j++)`	c_3	n
4	`if (v[j] <= pivo)`	c_4	$n-1$
5	`i++`	c_5	t
6	`trocar(v, i, j)`	c_6	t
7	`trocar(v, i + 1, fim)`	c_7	1
8	`return i + 1`	c_8	1

O teste do `for` é executado n vezes ($n-1$ vezes verdadeiro e uma vez falso, na saída do laço). A variável t é a quantidade de elementos de `v[ini..fim-1]` menores ou iguais ao pivô, com $0 \leq t \leq n-1$. Somando tudo:

$$P(n) = c_1 + c_2 + c_3 n + c_4 (n-1) + (c_5 + c_6)t + c_7 + c_8$$

No melhor caso o `if` nunca é verdadeiro ($t = 0$), o que acontece quando o pivô é o menor elemento:

$$P(n) = (c_3 + c_4)n + (c_1 + c_2 + c_7 + c_8 - c_4) = an + b$$

No pior caso o `if` é sempre verdadeiro ($t = n - 1$), quando o pivô é o maior elemento:

$$P(n) = (c_3 + c_4 + c_5 + c_6)n + (c_1 + c_2 + c_7 + c_8 - c_4 - c_5 - c_6) = a'n + b'$$

Nos dois casos $P(n)$ é uma função linear de n. Pela definição de Θ, precisamos de constantes $k_1, k_2 > 0$ e n_0 tais que $k_1 n \le P(n) \le k_2 n$ para todo $n \ge n_0$. Como todos os termos de $P(n)$ são não negativos, $P(n) \ge c_3 n$, então $k_1 = c_3$. Como $t < n$ e $c_i \le c_i n$ para $n \ge 1$, temos $P(n) \le (c_1 + c_2 + \cdots + c_8)n$, então $k_2 = c_1 + \cdots + c_8$. As duas desigualdades valem para $n \ge 1$, logo $n_0 = 1$ e

$$c_3 n \le P(n) \le (c_1 + \cdots + c_8)n \quad \Rightarrow \quad P(n) = \Theta(n)$$

Somando a escolha do pivô ($\Theta(1)$ na versão aleatória e $\Theta(n)$ na busca da versão proporcional), cada chamada do QuickSort gasta $\Theta(n)$ fora das chamadas recursivas. Esse é o termo cn que aparece nas recorrências a seguir.

3.2 ii. QuickSort com pivô aleatório (caso médio)

Recorrência. Como o pivô é sorteado entre n elementos distintos, o lado esquerdo pode ficar com $k = 0, 1, \ldots, n-1$ elementos, cada valor com probabilidade $1/n$, e o lado direito fica com $n - 1 - k$. Fazendo a média dos casos:

$$T(n) = \frac{1}{n} \sum_{k=0}^{n-1} [T(k) + T(n-1-k)] + cn, \qquad T(0) = T(1) = 0$$

Junção dos somatórios. Separando a soma em duas:

$$\sum_{k=0}^{n-1} T(k) = T(0) + T(1) + \cdots + T(n-1)$$

$$\sum_{k=0}^{n-1} T(n-1-k) = T(n-1) + T(n-2) + \cdots + T(0)$$

As duas têm os mesmos termos em ordem inversa (quando k vai de 0 a $n-1$, $n-1-k$ vai de $n-1$ a 0), então são iguais e

$$T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + cn \qquad (1)$$

Eliminação do somatório. Multiplicando (1) por n:

$$nT(n) = 2 \sum_{k=0}^{n-1} T(k) + cn^2 \qquad (2)$$

Escrevendo a mesma equação para $n - 1$:

$$(n-1)T(n-1) = 2 \sum_{k=0}^{n-2} T(k) + c(n-1)^2 \qquad (3)$$

Subtraindo (3) de (2), os somatórios se cancelam, sobrando apenas o termo $k = n - 1$:

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + c[n^2 - (n-1)^2] = 2T(n-1) + c(2n-1)$$
$$nT(n) = (n+1)T(n-1) + c(2n-1)$$

Dividindo por $n(n+1)$:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{c(2n-1)}{n(n+1)} \qquad (4)$$

Telescopagem. Fazendo $S(n) = \frac{T(n)}{n+1}$, a equação (4) fica $S(n) = S(n-1) + c\frac{2n-1}{n(n+1)}$. Aplicando a mesma relação para $S(n-1)$, $S(n-2)$ e assim por diante até $S(1)$:

$$\begin{aligned} S(n) &= S(n-1) + c\tfrac{2n-1}{n(n+1)} \\ &= S(n-2) + c\tfrac{2(n-1)-1}{(n-1)n} + c\tfrac{2n-1}{n(n+1)} \\ &\;\;\vdots \\ &= S(1) + c\textstyle\sum_{i=2}^{n} \frac{2i-1}{i(i+1)} \end{aligned}$$

Os termos intermediários se cancelam e sobram $S(1) = T(1)/2 = 0$ e o somatório.

Série harmônica. Por frações parciais,

$$\frac{2i-1}{i(i+1)} = \frac{3}{i+1} - \frac{1}{i}$$

o que pode ser conferido somando as frações: $\frac{3i-(i+1)}{i(i+1)} = \frac{2i-1}{i(i+1)}$. Usando a série harmônica $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$:

$$\sum_{i=2}^{n} \frac{3}{i+1} = 3(H_{n+1} - 1 - \frac{1}{2}), \qquad \sum_{i=2}^{n} \frac{1}{i} = H_n - 1$$

Como $H_{n+1} = H_n + \frac{1}{n+1}$:

$$\sum_{i=2}^{n} \frac{2i-1}{i(i+1)} = 2H_n + \frac{3}{n+1} - \frac{7}{2}$$

Voltando para $T(n) = (n+1)S(n)$ e mantendo o termo $c(2n-1)$ até o fim:

$$T(n) = c(n+1)(2H_n + \frac{3}{n+1} - \frac{7}{2}) = 2c(n+1)H_n - \frac{7}{2}c(n+1) + 3c$$

Limites de H_n. Para mostrar que H_n cresce como $\log n$, cada parcela $1/i$ é comparada com a área sob a curva $1/x$, que é decrescente. No intervalo $[i-1, i]$, com $i \geq 2$, vale $1/x \geq 1/i$, então $\frac{1}{i} \leq \int_{i-1}^{i} \frac{dx}{x}$; somando de $i = 2$ até n, $H_n \leq 1 + \int_1^n \frac{dx}{x} = 1 + \ln n$. No intervalo $[i, i+1]$ vale $1/x \leq 1/i$, então $\frac{1}{i} \geq \int_i^{i+1} \frac{dx}{x}$; somando de $i = 1$ até n, $H_n \geq \int_1^{n+1} \frac{dx}{x} = \ln(n+1)$. Portanto

$$\ln(n+1) \leq H_n \leq 1 + \ln n$$

Resultado. Usando $H_n \leq 1 + \ln n$:

$$T(n) \leq c(n+1)(2 + 2\ln n - \frac{7}{2}) + 3c = 2c(n+1)\ln n - \frac{3}{2}c(n+1) + 3c \leq 2c(n+1)\ln n \leq 4cn\ln n$$

para $n \geq 1$, já que $-\frac{3}{2}c(n+1) + 3c \leq 0$ e $n + 1 \leq 2n$. Logo $T(n) = O(n \log n)$.

Usando $H_n \geq \ln(n+1)$ e desprezando o $+3c$:

$$T(n) \geq c(n+1)(2\ln(n+1) - \frac{7}{2})$$

Para $n \geq 33$ temos $\ln(n+1) \geq \ln 34 \approx 3{,}53 \geq 3{,}5$, então $2\ln(n+1) - \frac{7}{2} \geq \ln(n+1)$ e $T(n) \geq c(n+1)\ln(n+1) \geq cn \ln n$. Logo $T(n) = \Omega(n \log n)$.

Como $cn \ln n \leq T(n) \leq 4cn \ln n$ para $n \geq 33$, conclui-se que, no caso médio, $T(n) = \Theta(n \log n)$. Mais precisamente, $T(n) \approx 2cn \ln n = 2 \ln 2 \cdot cn \log_2 n \approx 1{,}39\, cn \log_2 n$.

Validade para qualquer entrada. Na versão com pivô aleatório, a aleatoriedade vem do sorteio feito pelo algoritmo e não da ordem dos dados. Para qualquer vetor de entrada, ordenado, invertido ou não, o pivô sorteado em um trecho de tamanho m tem probabilidade $1/m$ de ser o 1º, o 2º, …, o m-ésimo menor elemento. Assim a recorrência acima vale para todas as entradas, e nenhuma entrada específica força o algoritmo a ter mau desempenho. Isso não acontece quando o pivô é fixo (por exemplo, sempre o último), pois nesse caso um vetor já ordenado leva sempre ao pior caso.

Pior caso. O pior caso ainda pode ocorrer, porque o sorteio pode escolher sempre o menor ou o maior elemento. Nesse caso as divisões são 0 e $n-1$ e o tempo é $\Theta(n^2)$ (ver o final do item iii). Porém isso depende de vários sorteios ruins seguidos, o que é muito improvável, e não depende da entrada.

3.3 iii. QuickSort Proporcional ($\alpha = 1/2$ e $\alpha = 1/4$)

Origem da recorrência. Em qualquer versão, depois de particionar um trecho de n elementos o QuickSort ordena um lado com k elementos e o outro com $n-1-k$:

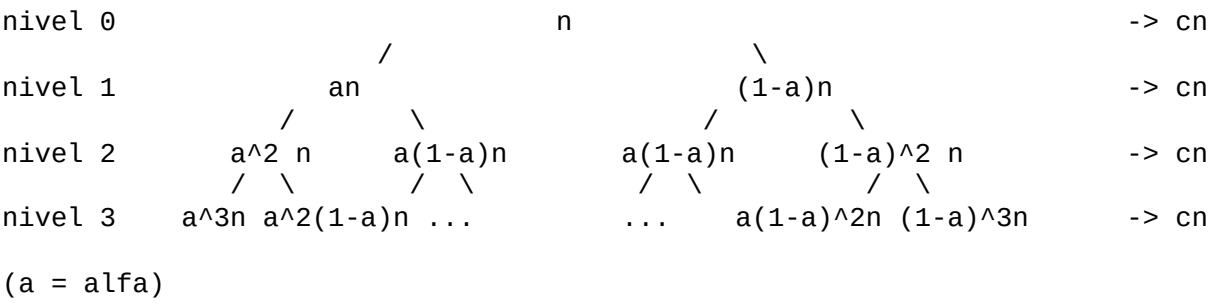
$$T(n) = T(k) + T(n-1-k) + cn$$

No Proporcional, $k = \lfloor \alpha(n-1) \rfloor \approx \alpha n$ e $n - 1 - k \approx (1-\alpha)n$, o que dá

$$T(n) = T(\alpha n) + T((1-\alpha)n) + cn, \qquad 0 < \alpha \leq 1/2$$

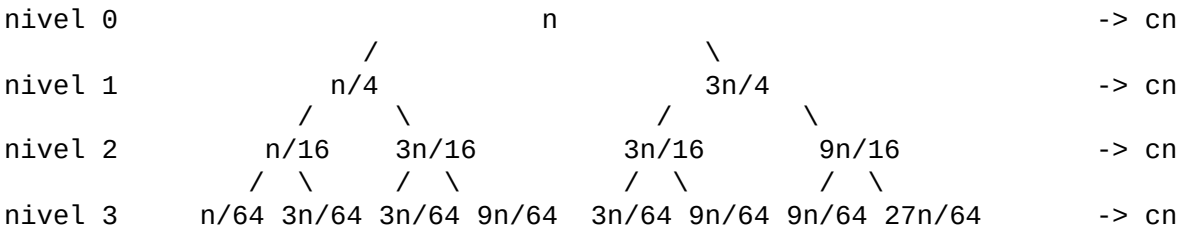
O -1 corresponde ao pivô, que sai do trecho, e o arredondamento para baixo altera o tamanho em menos de um elemento. Os tamanhos reais satisfazem $k \leq \alpha n$ e $n - 1 - k \leq (1-\alpha)n + 1$, ou seja, diferem de αn e $(1-\alpha)n$ por no máximo uma constante, enquanto os trechos têm tamanho proporcional a n. Uma constante somada ao tamanho não muda a ordem de grandeza da solução, pelo mesmo motivo que pisos e tetos costumam ser ignorados em recorrências. Além disso, sem o -1 os trechos ficam maiores, então a recorrência simplificada é um limite superior para a real.

Árvore de recursão. Cada nó representa um trecho, e o custo do nó é c vezes o seu tamanho. Para um α qualquer:



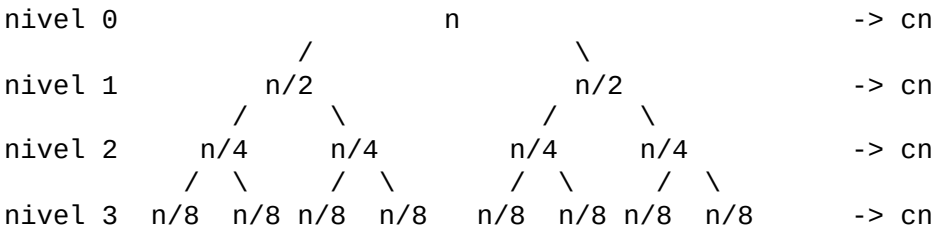
No nível 3 há um trecho de tamanho $\alpha^3 n$, três de $\alpha^2(1-\alpha)n$, três de $\alpha(1-\alpha)^2 n$ e um de $(1-\alpha)^3 n$, que são os termos de $(\alpha + (1-\alpha))^3 = 1$. Assim, a soma dos tamanhos de um nível i é $n(\alpha + (1-\alpha))^i = n$, já que cada nível apenas reparte os mesmos elementos. Todo nível completo custa cn, e os níveis incompletos, abaixo do fim do ramo mais curto, custam no máximo cn.

Para $\alpha = 1/4$:



Conferindo o nível 3: $\frac{1+3+3+9+3+9+9+27}{64} = \frac{64}{64} = 1$, ou seja, a soma é n.

Para $\alpha = 1/2$:



Com $\alpha = 1/2$ os dois lados são iguais em todos os níveis, a árvore é equilibrada e todos os ramos têm o mesmo comprimento, portanto $h_{min} = h_{max}$.

Alturas h_{min} e h_{max}. O ramo mais curto segue sempre o lado menor, com tamanho $\alpha^i n$ no nível i, e termina quando o trecho chega a 1:

$$n\alpha^{h_{min}} = 1 \;\Rightarrow\; \left(\frac{1}{\alpha}\right)^{h_{min}} = n \;\Rightarrow\; h_{min} = \log_{1/\alpha} n$$

O ramo mais longo segue sempre o lado maior, com tamanho $(1-\alpha)^i n$:

$$n(1-\alpha)^{h_{max}} = 1 \;\Rightarrow\; h_{max} = \log_{1/(1-\alpha)} n$$

Para $\alpha = 1/2$, $h_{min} = h_{max} = \log_2 n$. Para $\alpha = 1/4$, $h_{min} = \log_4 n = \frac{1}{2}\log_2 n$ e $h_{max} = \log_{4/3} n$.

Limites inferior e superior. Do nível 0 até o nível h_{min} todos os níveis estão completos e custam cn cada. Desprezando o restante da árvore:

$$T(n) \geq cn \cdot h_{min} = cn \log_{1/\alpha} n \;\Rightarrow\; T(n) = \Omega(n \log n)$$

A árvore tem no máximo $h_{max} + 1$ níveis (do 0 ao h_{max}), cada um com custo no máximo cn:

$$T(n) \leq cn(h_{max} + 1) = cn \log_{1/(1-\alpha)} n + cn \;\Rightarrow\; T(n) = O(n \log n)$$

Alturas na forma $k \cdot \log_2 n$. Pela mudança de base, $\log_b n = \frac{\log_2 n}{\log_2 b}$, então

$$h_{min} = \frac{1}{\log_2 (1/\alpha)} \log_2 n = k_{min} \log_2 n, \qquad k_{min} = \frac{1}{\log_2 (1/\alpha)}$$

$$h_{max} = \frac{1}{\log_2 \frac{1}{1-\alpha}} \log_2 n = k_{max} \log_2 n, \qquad k_{max} = \frac{1}{\log_2 \frac{1}{1-\alpha}}$$

Para $\alpha = 1/2$: $k_{min} = k_{max} = \frac{1}{\log_2 2} = 1$.

Para $\alpha = 1/4$: $k_{min} = \frac{1}{\log_2 4} = \frac{1}{2}$ e $k_{max} = \frac{1}{\log_2 (4/3)} = \frac{1}{2 - \log_2 3} \approx \frac{1}{0{,}415} \approx 2{,}41$.

Com isso os limites ficam

$$c\,k_{min}\, n \log_2 n \le T(n) \le c\,k_{max}\, n \log_2 n + cn$$

Para $n = 1.000.000$ ($\log_2 n \approx 20$), com $\alpha = 1/2$ a árvore tem cerca de 20 níveis; com $\alpha = 1/4$ o ramo mais curto tem cerca de 10 níveis e o mais longo cerca de 48.

Conclusão. Para $\alpha = 1/2$, $cn \log_2 n \le T(n) \le cn \log_2 n + cn$, logo $T(n) = \Theta(n \log n)$; este é o melhor caso possível do QuickSort. Para $\alpha = 1/4$, $\frac{c}{2} n \log_2 n \le T(n) \le 2{,}41\, cn \log_2 n + cn$, logo também $T(n) = \Theta(n \log n)$, mas com constante maior. De forma geral, para qualquer α constante com $0 < \alpha \le 1/2$ os dois limites são uma constante vezes $n \log n$, e portanto $T(n) = \Theta(n \log n)$. O valor de α altera apenas as constantes k_{min} e k_{max}: quanto menor α, mais alta e desbalanceada é a árvore, mas a ordem de grandeza continua a mesma enquanto a divisão for proporcional.

Partição não proporcional. Se a divisão for sempre 0 e $n - 1$ (pivô sempre o menor ou o maior elemento), o lado menor deixa de ser uma fração de n e passa a ter tamanho constante:

$$T(n) = T(n-1) + T(0) + cn = T(n-1) + cn$$

Expandindo:

$$T(n) = cn + c(n-1) + c(n-2) + \cdots + 2c = c(\frac{n(n+1)}{2} - 1) = \Theta(n^2)$$

Nesse caso a árvore se reduz a um caminho de altura $n - 1$, e o custo dos níveis diminui (cn, $c(n-1)$, …) em vez de ser cn em todos. A altura passa de $\Theta(\log n)$ para $\Theta(n)$, porque o tamanho do trecho diminui de 1 em 1 em vez de ser dividido por uma constante, e o tempo passa de $\Theta(n \log n)$ para $\Theta(n^2)$. Sem o tratamento descrito na seção 2, a pilha chegaria a n chamadas e ocorreria `StackOverflowError`. O mesmo vale para qualquer divisão em que o lado menor tenha tamanho constante, como 1 e $n - 2$.

3.4 Quadro-resumo

Versão	**Recorrência**	**Melhor caso**	**Caso médio**	**Pior caso**	**Pilha**
Particionamento (Lomuto)	—	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(1)$
Pivô aleatório	$T(n) = \frac{2}{n}\sum_{k=0}^{n-1} T(k) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$*	$O(\log n)$
Proporcional, $\alpha = 1/2$	$T(n) = 2T(n/2) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$
Proporcional, $\alpha = 1/4$	$T(n) = T(n/4) + T(3n/4) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$
Proporcional, α constante	$T(n) = T(\alpha n) + T((1-\alpha)n) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$
Partição 0 e $n-1$	$T(n) = T(n-1) + cn$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$**

* Muito improvável e independente da entrada. ** Sem o tratamento de recursão no lado menor seria $O(n)$.

α	h_{min}	k_{min}	h_{max}	k_{max}
1/2	$\log_2 n$	1	$\log_2 n$	1
1/4	$\frac{1}{2}\log_2 n$	1/2	$\log_{4/3} n$	$\approx$2,41

Na versão Proporcional a divisão é imposta pelo próprio algoritmo, por isso, para um α fixo, o melhor caso, o caso médio e o pior caso são iguais.

4 REFERÊNCIAS

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

ZIVIANI, N. **Projeto de algoritmos com implementações em Java e C++**. São Paulo: Thomson Learning, 2007.